CI/CD Pipeline for Python Energy Management Application using Docker

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	**ARNIKA JAIN**	**EN22CS301196**
02	**ANSH UPADHYAY**	**EN22CS301147**
03	**ARYAN SINGH**	**EN22CS301214**
04	**ANUROOP PATIDAR**	**EN22CS301176**
05	**ARNODAY SONI**	**EN22CS301197**

Group Name: Group 01-D2

Project Number:DO-13

Industry Mentor Name:

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2026

MEDICAPS UNIVERSITY

Datagami
"Lead Digital Technology"

TABLE OF CONTENTS

S.NO.	TOPICS	
1	INTRODUCTION	1.1 SCOPE OF THE DOCUMENT
		1.2 INTENDED AUDIENCE
		1.3 SYSTEM OVERVIEW
2	SYSTEM DESIGN	2.1 APPLICATION DESIGN
		2.2 PROCESS FLOW
		2.3 INFORMATION FLOW
		2.4 COMPONENTS DESIGN
		2.5 KEY DESIGN CONSIDERATIONS
		2.6 API CATALOGUE
3.	DATA DESIGN	3.1 DATA MODEL
		3.2 DATA ACCESS MECHANISM
		3.3 DATA RETENTION POLICIES
		3.4 DATA MIGRATION
4.	INTERFACES	
5.	STATE AND SESSION MANAGEMENT	
6.	CACHING	
7.	FUNCTIONAL REQUIREMENTS	7.1 AUTOMATED BUILD AND TESTING PROCESS
		7.2 APPLICATION CONTAINERIZATION USING DOCKER
		7.3 DEPLOYMENT ON AWS EC2
		7.4 MONITORING AND LOGGING
8.	NON-FUNCTIONAL REQUIREMENTS	8.1 SECURITY

		8.2 PERFORMANCE
		8.3 SCALABILITY
		8.4 RELIABILITY
9.	**REFERENCES**	

1. INTRODUCTION

Modern AI-based applications require fast, reliable, and automated deployment pipelines. This project focuses on building a **CI/CD pipeline for an AI-powered Energy Management Application** that uses Agentic AI and Retrieval-Augmented Generation (RAG) techniques.

The system integrates DevOps tools such as Git, Docker, Jenkins/GitHub Actions, and Kubernetes to automate the entire lifecycle — from code integration to deployment.

The application enables users to perform energy-related research, upload documents, and generate AI-based structured reports. The CI/CD pipeline ensures continuous delivery, scalability, and reliability.

1.1 SCOPE OF THE DOCUMENT

This document provides a High-Level Design of the system, including:

- Overall architecture of the AI-based energy research platform
- Integration of DevOps tools
- Workflow of CI/CD pipeline
- Deployment strategy using Docker and Kubernetes
- High-level interaction between system components

This document does not cover low-level implementation details.

1.2 INTENDED AUDIENCE

This document is intended for:

- Project evaluators
- DevOps engineers
- Software developers
- System architects
- Administrators managing deployment

1.3 SYSTEM OVERVIEW

The system automates the complete software delivery lifecycle of the Energy Management Application. Developers push the application code to a GitHub repository, which triggers the CI/CD pipeline.The pipeline performs build, testing, and containerization using Docker, and finally deploys the application to AWS.This automated process ensures faster delivery, consistency across environments, and improved reliability of the system.

2. SYSTEM DESIGN

The system design follows a modular three-layer architecture consisting of a Streamlit-based frontend, a FastAPI backend, and a FAISS-based data layer for storing embeddings. It integrates an Agentic AI system using LangChain or CrewAI to process user queries, perform analysis, and generate structured reports using RAG and LLMs. The application is containerized using Docker and deployed through a CI/CD pipeline with tools like GitHub Actions or Jenkins, and managed using Kubernetes for scalability and reliability. This design ensures automation, consistency, and efficient performance across the system.

2.1 APPLICATION DESIGN

The application design follows a three-layer architecture consisting of the presentation, application, and data layers. The presentation layer is built using Streamlit, which provides a user-friendly interface for submitting queries, uploading documents, and viewing AI-generated reports. The application layer is developed using FastAPI and handles the core logic, including processing user requests, managing API communication, and coordinating AI agents. The data layer uses a FAISS vector database to store document embeddings and support efficient retrieval for RAG.

2.2 PROCESS FLOW

The process flow begins when a user submits a research query through the Streamlit interface. This query is sent to the FastAPI backend, which processes the request and triggers the Agentic AI system. Multiple AI agents then perform tasks such as searching for relevant information, extracting data, and analyzing content. The system retrieves related documents from the FAISS vector database using RAG. The LLM then generates a structured report based on the gathered information. Finally, the generated report is sent back to the frontend and displayed to the user.

2.3 INFORMATION FLOW

1. User submits a query through the Streamlit interface
2. The query is sent to the FastAPI backend
3. Backend forwards the request to the Agentic AI system
4. AI agents process the query (search, analysis, extraction)
5. Relevant data is retrieved from the FAISS vector database
6. LLM generates a structured response/report
7. The response is sent back to the backend
8. Backend returns the final output to the user interface
9. User views the generated report on the frontend

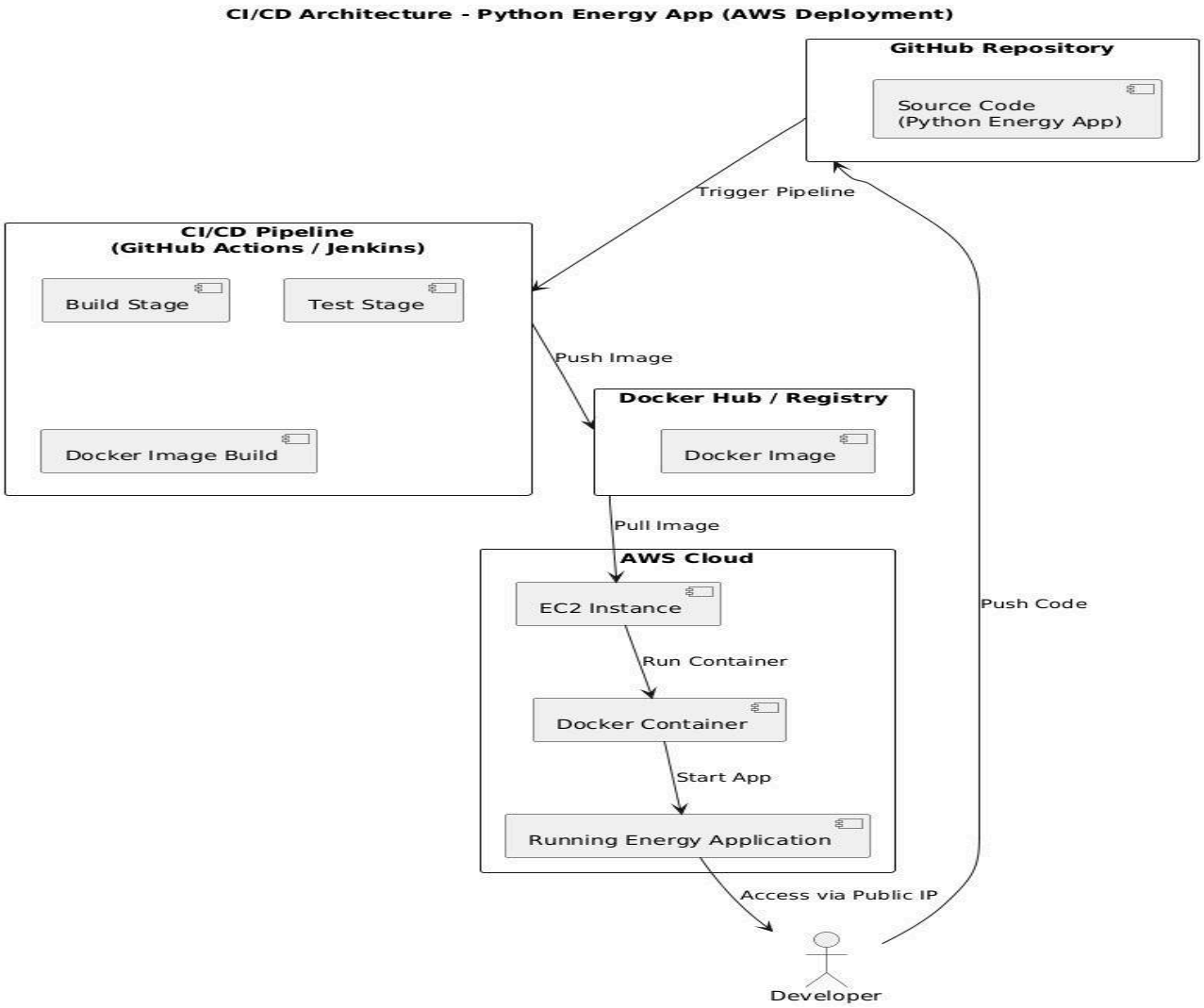

Figure 1 :High-Level System Architecture

2.4 COMPONENTS DESIGN

1. GitHub Repository stores the source code and triggers the pipeline.
2. CI/CD Pipeline automates build, testing, and deployment processes.
3. Docker containerizes the application and ensures consistency.
4. Docker Registry stores versioned Docker images.
5. AWS manages deployment, scaling, and availability.

These components work together to provide an automated and scalable system.

2.5 KEY DESIGN CONSIDERATIONS

1. Modularity ensures separation of components for better maintainability.
2. Scalability is achieved using Kubernetes auto-scaling features.
3. Reliability is maintained through automated testing and deployment.
4. Portability is ensured using Docker containers.

5. Automation reduces manual effort and human errors.

2.6 API CATALOGUE

1. AWS API is used by the CI/CD pipeline to manage deployments.
2. It helps in creating, updating, and scaling pods and services.

3. DATA DESIGN

The data design of the CI/CD pipeline for the Energy Management Application focuses on managing and tracking information across different stages of the software delivery lifecycle. The system maintains structured data related to source code repositories, pipeline executions, Docker images, and deployment instances on AWS. This data helps in ensuring traceability, monitoring, and efficient management of the application from development to deployment.

3.1 DATA MODEL

1. The system includes entities such as Repository, Pipeline Job, Docker Image, and Deployment Instance.
2. These entities represent different stages of the CI/CD pipeline.

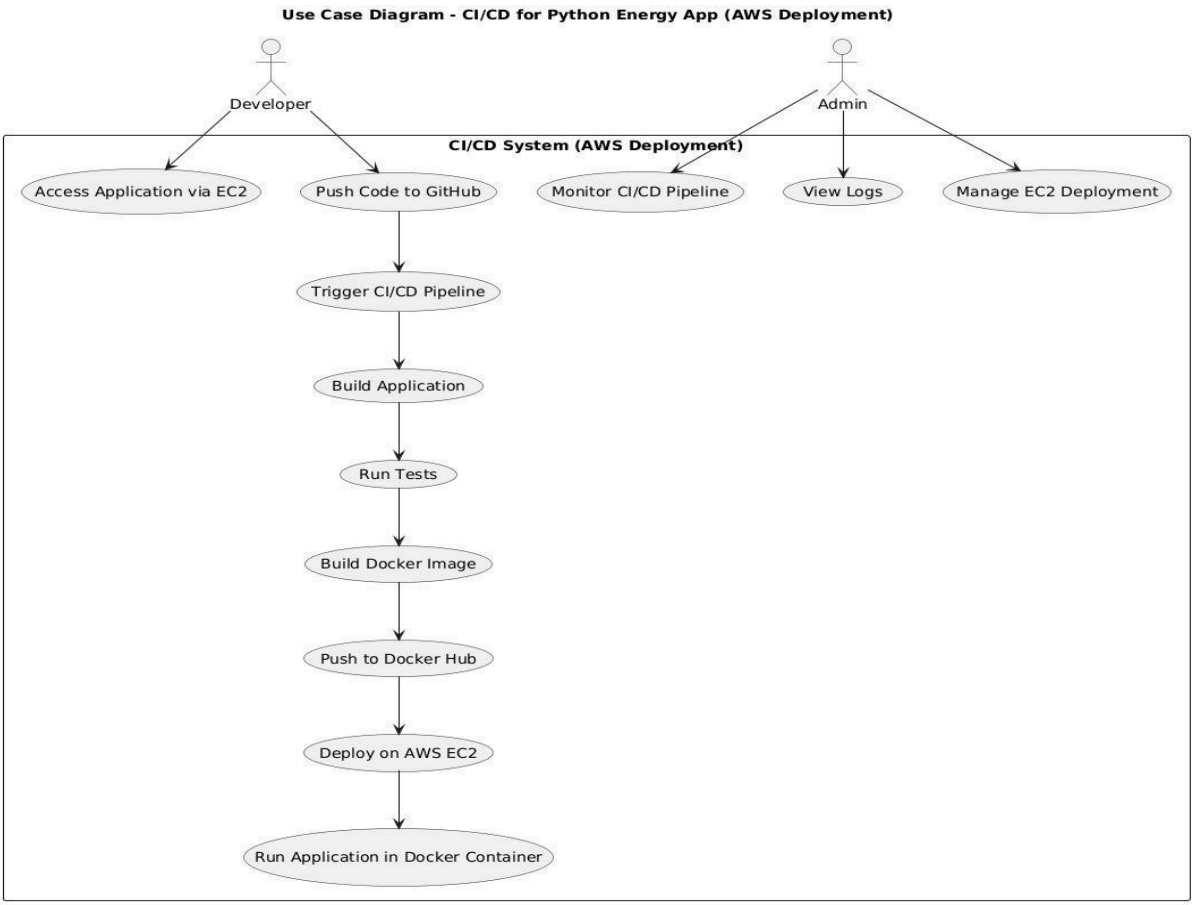

Figure 2 :Use Case Diagram

Logical Relationships (The "Lines")

1. Actors in the System

- **Developer**
  - Responsible for writing code
  - Pushes code to GitHub repository
  - Accesses the deployed application via AWS EC2
- **Admin**
  - Monitors the CI/CD pipeline
  - Views logs for build and deployment
  - Manages deployment on AWS EC2

2. Developer Interactions

- Developer pushes code to the GitHub repository
- This action automatically triggers the CI/CD pipeline
- The pipeline performs the following stages:
  - Building the application
  - Running tests
  - Creating a Docker image
  - Pushing the image to Docker Hub
- The application is deployed on AWS EC2
- Developer accesses the running application from EC2

3. CI/CD System Workflow

- The process starts when code is pushed to GitHub
- CI/CD pipeline is triggered automatically
- Application is built and tested
- Docker image is created
- Image is stored in Docker Hub
- AWS EC2 pulls the Docker image
- Application is deployed in a Docker container
- The application starts running on EC2

4. Admin Interactions

- Admin monitors the CI/CD pipeline status
- Admin checks logs for build and deployment processes
- Admin manages EC2 instances (start, stop, and update deployments)

3.2 DATA ACCESS MECHANISM

1. GitHub access is controlled using authentication tokens.
2. Docker registry requires secure credentials for access.
3. AWS resources are accessed using IAM roles and permissions.

3.3 DATA RETENTION POLICIES

1. Pipeline logs are stored for monitoring and debugging.
2. Docker images are versioned for rollback purposes.
3. AWS maintains deployment logs and instance data.

3.4 DATA MIGRATION

1. New application versions create new Docker images.
2. AWS deploys updated containers on EC2 instances.
3. Older versions can be restored when required.

4. INTERFACES

The system provides multiple interfaces for interaction between users and the CI/CD pipeline. These interfaces also enable communication between different system components and ensure smooth operation, monitoring, and management of the application throughout its lifecycle.

4.1 USER INTERFACE

1. GitHub interface is used for code management.
2. Jenkins/GitHub Actions dashboard is used to monitor pipelines.
3. AWS Management Console is used to manage deployment.

4.2 COMMAND LINE INTERFACE

1. Git CLI is used for version control operations.
2. Docker CLI is used for building and running containers.
3. AWS CLI is used for managing cloud resources.
4. CLI tools provide flexibility for developers and engineers.

5. STATE AND SESSION MANAGEMENT

1. AWS EC2 maintains the running state of the application.
2. CI/CD pipeline tracks build and deployment stages.
3. Application sessions are managed using secure mechanisms.
4. This ensures system stability and availability.

6. CACHING

1. Docker layer caching improves build performance.
2. Dependency caching reduces installation time.
3. Pipeline artifacts are reused to optimize efficiency.
4. Caching improves overall system speed and performance.

7. FUNCTIONAL REQUIREMENTS

1. Functional requirements define the core features of the CI/CD system.
2. They ensure automated development, testing, containerization, and deployment.
3. These requirements describe system behavior and services provided to users.

7.1 AUTOMATED BUILD AND TESTING PROCESS

1. The system automatically builds the application when code is pushed.
2. It installs dependencies and prepares the application for execution.
3. It runs automated tests such as unit and integration testing.

7.2 APPLICATION CONTAINERIZATION USING DOCKER

1. The system packages the application into Docker containers.
2. A Dockerfile defines runtime environment and dependencies.
3. The container runs consistently across all environments.

7.3 DEPLOYMENT ON AWS EC2

1. The system deploys the application on AWS EC2 instances.
2. EC2 pulls Docker images from a registry.
3. The application runs inside Docker containers on EC2.

7.4 MONITORING AND LOGGING

1. The system monitors CI/CD pipeline execution.
2. Logs are generated for build, test, and deployment stages.
3. Logs are accessible for debugging and analysis.

8. NON-FUNCTIONAL REQUIREMENTS

1. These define quality attributes and system constraints.
2. They ensure efficiency, security, scalability, and reliability.

8.1 SECURITY

1. Secure authentication for GitHub and AWS access.
2. Use of IAM roles for access control.
3. Protection of sensitive data using environment variables.

8.2 PERFORMANCE

1. Fast automated deployment process.
2. Efficient utilization of AWS resources.
3. Reduced manual effort through automation.

8.3 SCALABILITY

1. AWS allows scaling of EC2 instances based on demand.
2. The system can handle increased workload efficiently.

8.4 RELIABILITY

1. Continuous monitoring of applications on AWS.
2. Automatic restart of services in case of failure.
3. Consistent deployment using CI/CD pipeline.

9. REFERENCES

[1] DevOps Course Material, *Medicaps University – Datagami Skill Based Course*, 2025–2026.

[2] Git Documentation, "Git Official Documentation," [Online]. Available: https://git-scm.com/docs

[3] Docker Documentation, "Docker Official Documentation," [Online]. Available: https://docs.docker.com

[4] Amazon Web Services Documentation, "AWS EC2 Documentation," [Online]. Available: https://docs.aws.amazon.com/ec2

[5] Jenkins Documentation, "Jenkins User Documentation," [Online]. Available: https://www.jenkins.io/doc

[6] GitHub Actions Documentation, "GitHub Actions Documentation," [Online]. Available: https://docs.github.com/actions

[7] Docker Hub Documentation, "Docker Hub Documentation," [Online]. Available: https://hub.docker.com

[8] Energy Management Application Project Repository, GitHub, 2026.